Ошибки и исключения

Перед выполнением изучить главу 09 «Ошибки и исключения»

Задание

Разработать программу импорта банковской выписки. Пути к файлам передаются через аргументы командной строки. Программа отделяет корректные операции от ошибочных строк и формирует отчет

Пример запуска:

```
./bank_import valid_statement.txt mixed_statement.txt
edge_cases_statement.txt
```

Для проверки использовать следующие входные файлы:

Файл	Содержимое
`valid_statement.txt`	корректная выписка
`mixed_statement.txt`	выписка с корректными и ошибочными строками
`edge_cases_statement.txt`	выписка с граничными случаями
`missing_statement.txt`	имя отсутствующего файла для проверки ошибки открытия

Файл содержит одну банковскую операцию на строку:

```
id;date;type;amount;description
```

Например:

```
TX1001;2026-09-01;TRANSFER;-1250.00;Taxi
TX1002;2026-09-01;PAYMENT;-249.90;Coffee
TX1003;2026-09-02;INCOME;75000.00;Salary
TX1004;2026-09-03;FEE;-99.00;
```

Типы операций

Тип	Правило суммы
`INCOME`	сумма больше `0`
`PAYMENT`	сумма меньше `0`
`TRANSFER`	сумма не равна `0`
`FEE`	сумма меньше `0`

Строка содержит ровно пять полей, разделенных `;`, без строки заголовка. Пустое описание сохраняет последний разделитель. Символ `;` внутри поля отсутствует, пробелы внутри описания сохраняются

Пустое поле `description` хранить как отсутствие значения через `std::optional`

Дата должна быть существующей календарной датой с годом от `0001` до `9999`, с учетом високосных лет

Сумма записывается десятичными цифрами с точкой и ровно двумя знаками после нее. Перед целой частью допустим знак `-`, например `-1250.00`. Пробелы,

знак +, экспоненциальная запись и другие разделители считаются ошибкой. Для точного расчета хранить суммы целым числом копеек. Превышение диапазона выбранного типа должно сообщаться как ошибка

Уникальность `id` проверяется отдельно для каждого файла среди уже принятых операций. Ошибочная строка оставляет идентификатор доступным для следующей корректной строки

Обработка ошибок

Ошибки строки: пустая строка, неверное количество полей, пустой или повторный `id`, ошибочная дата, неизвестный тип, неверный формат или знак суммы. Сохранить номер строки и причину в отчете, затем продолжить импорт

При запуске без аргументов вывести сообщение с примером использования

Ошибка открытия файла должна вызывать исключение, связанное с `std::exception`. Перехватить его через `try` и `catch`, вывести причину и перейти к следующему файлу. Для проверки передать среди аргументов отсутствующий `missing_statement.txt`

Файловый поток использовать как RAII-объект. Для проверки внутренних предположений применить осмысленный `assert`, например проверку согласованности счетчиков отчета

Отчет

Для каждого обработанного файла вывести:

- имя файла
- количество строк, принятых операций и ошибок
- список ошибок с номером строки и причиной
- итоговый баланс, сумму доходов и сумму расходов

Начальный баланс равен нулю. Доходы - сумма всех положительных операций, включая входящие `TRANSFER`. Расходы - положительная величина суммы всех отрицательных операций, включая исходящие `TRANSFER`. Баланс равен доходам минус расходы

Если добавление операции вызывает переполнение итогов, строка попадает в отчет об ошибках, итоги сохраняют прежние значения

Требования

- Названия сущностей записывать на английском языке без транслитерации
- Разбить программу как минимум на один `.hpp` и два `.cpp` файла. Для headers использовать `#pragma once`
- Программа должна компилироваться и выполнять предусмотренные сценарии
- Студент должен понимать код, обосновывать решение и отвечать на вопросы при защите

Минимальный набор функций: `parseOperationType`, `parseDate`, `parseAmount`, `parseTransaction`, `parseCommandLine`, `importStatement`, `validateTransaction`, `calculateBalance`, `printImportReport`